

Design Rationale

Note

The tutorial, lab and workbook contain several repeated questions. I have answered these questions once in a single PDF. Questions unique to each document are answered in separate Markdown files.

Why I chose these classes

I chose Patient, Practitioner, and Appointment because they represent the main people in SmartCare's booking brief.

- Patient stores information about the person receiving care.
- Practitioner stores information about the person providing care.
- Appointment stores the booking details and connects a patient with a practitioner.

How the classes connect

Appointment connects only to one patient and only one practitioner. A patient can have many appointments, and a practitioner can have many appointments. Both can also have no appointments. (Many-to-one relationship for both practitioner and patient towards appointment).

Why I separated the information

Keeping patient and practitioner details separate from appointments avoids copying the same information into every booking. It also makes details easier to update.

How the design supports the code

Each class has a clear purpose. The Python class skeletons follow the same structure UML, making it easier to add the working code later.